

International Journal of Geographical Information Science

ISSN: (Print) (Online) Journal homepage: www.tandfonline.com/journals/tgis20

Online discovery of co-movement patterns in mobility data

Andreas Tritsarolis, George-Stylianos Theodoropoulos & Yannis Theodoridis

To cite this article: Andreas Tritsarolis, George-Stylianos Theodoropoulos & Yannis Theodoridis (2021) Online discovery of co-movement patterns in mobility data, International Journal of Geographical Information Science, 35:4, 819-845, DOI: 10.1080/13658816.2020.1834562

To link to this article: https://doi.org/10.1080/13658816.2020.1834562

Published online: 23 Nov 2020.

Submit your article to this journal

Article views: 635

View related articles

View Crossmark data

Citing articles: 2 View citing articles

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=tgis20

INTERNATIONAL JOURNAL OF GEOGRAPHICAL INFORMATION SCIENCE
2021, VOL. 35, NO. 4, 819–845
https://doi.org/10.1080/13658816.2020.1834562

Taylor & Francis
Taylor & Francis Group

RESEARCH ARTICLE

Check for updates

Online discovery of co-movement patterns in mobility data

Andreas Tritsarolis, George-Stylianos Theodoropoulos and Yannis Theodoridis

Data Science Labouratory, Department of Informatics, University of Piraeus, Piraeus, Greece

ABSTRACT

The advent of GPS technologies generates location data-streams and accentuates the importance of developing practical tools that can process and analyze the vast amounts of location data at a given moment in a meaningful way. Profiling the trajectory of a moving object with respect to the trajectories of its surrounding objects, for example, can elicit its mobility behaviour and analyze it in order to inform domain experts with critical knowledge in real time. For instance, clustering multiple moving objects with respect to their spatial and temporal dimension to identify co-movement patterns. In this paper, we propose a novel graph-based online co-movement pattern mining algorithm, called *EvolvingClusters*, which can be used to discover different collective movement behaviours (like the well-known flocks and convoys) in a unified way based on the activity of multiple concurrent objects through time and space. We evaluate *EvolvingClusters* using real-world and synthetic data-sets from multiple mobility domains. Our study demonstrates the effectiveness of the proposed algorithm as well as its value towards a tool to profile semantically rich behaviour and with capabilities to observe and categorize multiple moving objects in real-time.

ARTICLE HISTORY
Received 1 March 2020
Accepted 6 October 2020

KEYWORDS
Data analytics; semantic enrichment; collective movement behaviour; co-movement patterns; evolving clusters

1. Introduction

Mobility Data Analytics (Giannotti and Pedreschi 2008, Andrienko *et al*. 2013, Pelekis and Theodoridis 2014) is a growing branch of the general spectrum of Data Science. GPS enabled mobile phones as well as specialized equipment on board of cars, aircrafts and vessels, are the most common sources, broadcasting huge volumes of location data. Using the latter as-is (i.e. in its 'raw' form) offers limited usefulness to data scientists and domain experts. Thus, proper processing (cleansing, transformation, enrichment, simplification, etc.) and analysis (pattern discovery, behavioural profiling, etc.) are vital tools that aid us in understanding and making effective use of the available information.

Grouping objects together based on the mobility behaviour that can be inferred from the data points generated by them, for example, can provide us with a useful background that can benefit other analytics techniques in multiple ways. Some relevant techniques include:

- *Object Profiling*, i.e., creating models that are mapped to multiple different objects with similar behaviour;

CONTACT Andreas Tritsarolis andrewt@unipi.gr
© 2020 Informa UK Limited, trading as Taylor & Francis Group

- *Trajectory compression/simplification*, by using the aforementioned models as points of reference that are able to adequately describe multiple trajectories;
- *Classification*, by using a more diverse set of trajectories/objects that are sampled from multiple of the available models, therefore reducing training times as well as preventing overfitting caused by a disproportionate amount of samples from one or more classes.

Taking into account the spatial proximity and direction of moving objects, *Flocks* (Benkert *et al*. 2008, Gudmundsson and van Kreveld 2006, Vieira *et al*. 2009) consist of a minimum number of objects that are located nearby (in a disk of a given radius) for at least some time duration. *Companion* (Tang *et al*. 2012) and *gathering* (Zheng *et al*. 2014) are two flock variations, focusing on online/streaming applications. A *convoy* (Jeung *et al*. 2008a, 2008b, Orakzai *et al*. 2019) differentiates from flocks in the sense that the circular neighborhood considered in flocks is replaced by the notion of density-connection discovered by a density-based spatial clustering algorithm like DBSCAN (Ester *et al*. 1996). Convoy variations include *groups* (Li *et al*. 2013) *groups* (Lan *et al*. 2017). A *swarm* (Li *et al*. 2010) is a collection of moving objects as in the case of convoys above, with the main difference being in the sense that swarms do not require the temporal duration of objects' closeness to be consecutive.

This paper focuses on group pattern mining over mobility data as well. More specifically, we discover and evaluate different co-movement behaviours of moving objects in a unified and online way. As an indicative example, Figure 1 illustrates the movement of nine objects during five consecutive time steps. The core idea of the paper is that at each time step a graph is created with respect to the objects' spatial proximity. Based on the created graphs, we discover group patterns according to a certain type (e.g. Maximal Cliques).

The motivation behind this paper is to expand the horizons of co-movement pattern discovery methods, by studying the problem from a point of view different from related work. In particular, we can emulate the existing pattern types using a baseline structure (i.e. the objects' connectivity graph). For instance, the popular flock and convoy patterns can be simulated by Maximal Cliques (MC) and Maximal Connected Subgraphs (MCS), respectively, as it will be presented in detail later. Moreover, since the patterns to be discovered are reduced to graph properties, we can reuse the same graph in order to

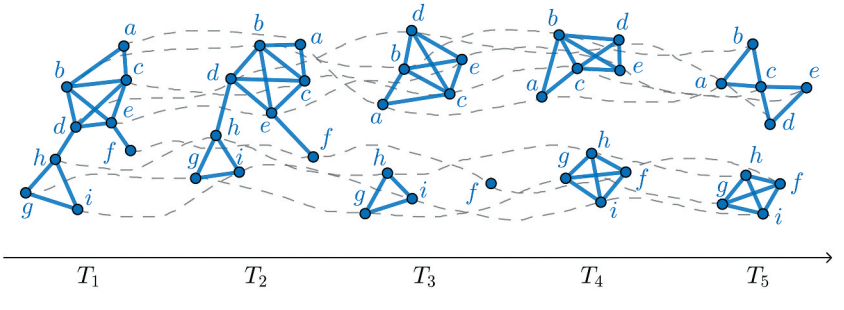


Figure 1. An example of nine objects moving at five consecutive timeslices and their respective connectivity graphs.

discover different clusters, at the same time. In particular, even though in this paper we discuss two graph properties (MC, MCS), by using the proposed graph-based approach to the group pattern discovery problem, we can further exploit on the discovered graphs, either by using other graph types (e.g. utility graphs), or by using graph metrics (e.g. centrality) in order to control the quality or the shape of the discovered patterns.

In a nutshell, our contributions are outlined as follows:

- We propose a novel group behaviour discovery algorithm, called *EvolvingClusters*, which is backed by a unified graph-based definition of what a group pattern is.
- We theoretically prove that our method is generic at a large extent simulating (as instantiations of it) the well-known flocks and convoys.
- We extensively evaluate the aforementioned algorithm using large-volume real-world data from multiple mobility domains, including maritime, aviation and urban data.

Our paper is structured as follows: Section 2 discusses the related work and how we put our work in there. Section 3 formulates the problem addressed in this paper. In Section 4, we propose our *EvolvingClusters* algorithm. Section 5 presents our experimental study and Section 6 further discusses some of our findings, related to each mobility domain of interest. Section 7 concludes the paper, also giving hints for future work.

2. Related work

The field of trajectory data analytics is rich in methods capturing collective movement of objects, i.e., sets of objects moving close to each other for a certain period of time.

The seminal works in the field include flocks, convoys and their variations, as discussed in Section 1. Focusing on more recent works, Matsubara *et al*. (2014) propose *StreamScan*, an algorithm that models streams of co-evolving sequences statistically, by discovering sub-sequences that have the same properties as a given Hidden Markov Model (HMM). The reference models are then used to categorize/cluster the data streams into groups/clusters that behave similarly.

Yadamjav *et al*. (2019) introduce a new variable to the group pattern mining domain, namely, the recurrence of an individual moving pattern. Then, they propose a methodology that is able to discover the so-called *recurrent convoys*.

Zhang *et al*. (2019) build a framework that is able to discover periodic spatio-temporal patterns that take into account both the irregularities in sampling that are frequently observed in real world data and the availability of background spatial information that can be vital in order to accurately interpret the semantic behaviour of a moving object. The main contributions of the paper include a novel trajectory representation that describes a trajectory as a sequence of semantic events that match spatial information, a method for discovering POIs with respect to apriori spatial, temporal, and semantic information, as well as a method for detecting regular time periods for each spatio-temporal concentration from irregular trajectories.

Sawas *et al*. (2019) discover pairs of pedestrians that spatially move together over a certain time interval, utilizing the discovered pairs as blocks to define larger groups. Thus, this method appears to be flexible by making use of versatile strategies and techniques.

The contributions of this work include *timeWgroups*, a method that is used to find groups of pedestrians with variant group pattern semantics, *individual perspective group pattern*, a pattern that represents the group of pedestrians that includes a specific pedestrian for at least a certain number of time steps, and *dominant groups* for pinpointing the most prevalent groups over a given time period.

Xian *et al*. (2019) introduce a parallel travelling pattern discovery method on heterogeneous data streams. The method partitions trajectories sampled on different time intervals, defines trajectory based distance metrics and discovers *Trajectory Companions* in a parallel/distributed environment. A proposed *Trajectory Slot Model* is used to handle the heterogeneous data streams and, alongside a *Coverage Density Reachability* metric, is able to partition the data stream spatially. A KD-tree index is also utilized to achieve a more load-balanced partitioning scheme. Since the data stream is transformed to polylines, a spatial distance metric is introduced to replace the closeness metric that is commonly used when handling coordinate points. Lastly, the aforementioned metric is used to discover the traveling patterns that are present in an online fashion.

Shein *et al*. (2020) describe an improved strategy for tracking loose group companions to avoid the strict requirement for members in a single group (i.e. Flocks, Convoys, etc.). In order to account for massive stream-based trajectory databases, their algorithm works in an incremental manner so as to minimize the computational complexity.

Helmi and Banaei-Kashani (2020) propose a frequent co-movement pattern (*f-CoMP*) definition for discovering patterns at multiple spatial scales, also exploiting the overall shape of the objects' trajectories, while at the same time it relaxes the temporal and spatial constraints of the seminal works (i.e. Flocks, Convoys, etc.) in order to discover more interesting patterns. Using a framework based on C#, they demonstrate the usability of *f-CoMP* in domains, such as sport and health analytics.

Zhao *et al*. (2020) propose a novel group pattern type, called 'converging', which is able to model converging behaviours of moving objects. Through extensive experimentation conducted on real-life urban trajectory datasets, they demonstrate the accuracy of the proposed framework (called, Converging Pattern Mining – CPM) as well as its perfomance superiority compared to related work (Guo *et al*. 2014).

Most related to our work, Fan *et al*. (2016) propose a general co-movement pattern definition that unifies those that are part of the past literature (Flocks, Convoys, etc.) and present a parallel and scalable framework able to discover the aforementioned patterns, deployed using Apache Spark. The proposed pattern discovery framework includes *Temporal Replication and Parallel Mining*, a method that, as suggested by its name, splits a data snapshot spatially and replicates data when necessary to ensure full coverage, and *Star Partitioning and ApRiori Enumerator*, a technique that uses graph pruning in order to avoid the data replication that takes place in the previous method.

Wiratma *et al*. (2019) evaluate four types of grouping techniques/definitions (namely, Original Groups, Refined Groups, Convoys and Swarms) with respect to three proposed metrics. These metrics represent how well each definition corresponds to the human intuition of a group, how the number of groups discovered by each proposed technique depends on the density of the objects in the specified environment, and how the number of discovered groups depends on the sampling rates that are used to form the needed trajectories.

Chen *et al*. (2019) and Fang *et al*. (2020) propose an online distributed co-movement pattern detection framework based on Apache Flink. The two-phase framework includes the clustering and the pattern enumeration phase, respectively. During the clustering phase for timestamp t_s, the snapshot S_t is clustered using Range-Join and DBSCAN. Additionally, in order to improve the performance of Range-Join, a two-layered GR-index (a global Grid index along with several local R-tree indexes, one per grid cell) is utilized inside of Flink. With the clustering phase complete for S_t, the authors propose and evaluate three alternative methods that can be used to discover all valid patterns at timestamp t.

Our approach in discovering co-movement/group patterns differs from the rest of the available literature in the ways that follow. First, our input does not assume temporal alignment and uniform sampling, assumptions that are very popular in related work although not valid in most cases of real world scenarios. Instead, our preprocessing pipeline evaluates alternative interpolation and extrapolation techniques to produce the correctly formatted output which will, in turn, be the input of our pattern discovery algorithm. Second, our algorithm is a modular/unified method that is able to discover different group patterns using a common underlying graph-based implementation. Third, to the best of our knowledge, it is the first work in the literature that is experimentally evaluated on three different domains (land, sea, and air transportation), providing insightful results.

3. Problem formulation

An informal definition for a *group pattern* could be: 'a large enough amount of objects moving along paths close to each other for a certain amount of time'. These objects could vary from animals (wolves, birds, lions, etc.) to human transportation means (e.g. cars, aircrafts and vessels). Discovering these patterns can provide insight regarding the behavior of these moving objects, from hunting (wolves, lions) and migration (birds) to traffic monitoring (transportation means) and fishing pressure (fishing vessels). In this paper, we aim at handling group pattern discovery in a uniform way, where 'closeness' is formulated using graph-based terminology.

Definition 1. (Evolving Cluster). Given: (i) a set T of moving objects, where the trajectory of each object consists of n pairs (p_i, t_i) of locations p_i recorded at timestamp t_i, (ii) a minimum cardinality threshold c, (iii) a maximum distance threshold θ, and (iv) a minimum time duration threshold d, an Evolving Cluster $\langle C, t_{start}, t_{end}, tp \rangle$ is a subset $C \subseteq T$ of the moving objects population, $|C| \geq c$, which appeared at timeslice t_{start} and remained alive until timeslice t_{end} (with $t_{end} - t_{start} \geq d$) during the lifetime t_{start}, t_{end} of which the participating moving objects were spatially connected with respect to distance θ and cluster type tp.

The abstract term 'spatially connected' is purposely used in the above definition, in order for it to account for a number of different clustering types tp. In this work for instance, we use spherical and density-based clustering in order to discover flock- and convoy-like patterns, respectively.

In particular, for each timeslice, let us consider the mapping of the points of moving object trajectories (that are active at that timeslice) in a connectivity graph $G(V, E)$, where vertex $v \in V$ represents a point p_i and edge $e \in E$ represents a pair of points (p_i, p_j) if and

only if $d(p_i, p_j) < \theta, p_i \neq p_j$; *Maximal Cliques (MCs)* in this graph correspond to spherical-like clusters whereas *Maximal Connected Subgraphs (MCSs)* correspond to density-connected clusters. MCs (MCSs) that remain alive for an adequate period of time are evolving clusters, according to the above definition, resembling flock- (respectively, convoy-) like patterns. This concept is better illustrated, if we recall Figure 1, where clusters:

- $C_1 = \{a, b, c, d, e, f, g, h, i\}$ and $C_6 = \{g, h, f, i\}$ form an MCS and an MC, respectively, that remain active for a period of two timeslices (T_1, T_2 and T_4, T_5, respectively).
- $C_3 = \{b, c, d, e\}$ forms an MC that remains active for a period of four timeslices (T_1, T_4).
- $C_2 = \{a, b, c\}$, $C_4 = \{g, h, i\}$ and $C_5 = \{a, b, c, d, e\}$ form two MCs and an MCS, respectively, that remain active for a period of five timeslices (T_1, T_5).

Assuming thresholds $c = 3$ and $d = 2$, we have discovered the above six evolving clusters, four spherical-like $\langle C_2, T_1, T_5, 1\rangle$, $\langle C_3, T_1, T_4, 1\rangle$, $\langle C_4, T_1, T_5, 1\rangle$ and $\langle C_6, T_4, T_5, 1\rangle$ as well as two density-based $\langle C_1, T_1, T_2, 2\rangle$ and $\langle C_5, T_1, T_5, 2\rangle$, where $tp = 1$ (2) corresponds to MC (respectively, MCS).

On the other hand, assuming different thresholds, e.g. $c = 5$ and $d = 4$, we discover two evolving clusters, the spherical-like $\langle C_3, T_1, T_4, 1\rangle$ and the density-based $\langle C_5, T_1, T_5, 2\rangle$. It is clear from the above discussion that the output may significantly vary with respect to the input parameters, thus thresholds should be carefully set according to the mobility domain (see Section 5.2.1 for a more detailed discussion and experimentation on the threshold effect on the Evolving Cluster output).

Hence, the main objective addressed in this paper is as follows: given a data stream D of moving object locations, the goal is to discover two types of *Evolving Clusters*, namely, MCs and MCSs, in online mode, without any assumption on the sampling rate of the transmitted locations. In the above discussion (c.f. Figure 1), we have assumed fixed (i.e. $|T_2 - T_1| = |T_3 - T_2|$, and so on) and synchronized sampling rate (i.e. all objects broadcast their location at the same timestamp). However, this assumption is rare in real-world cases, therefore, it will be relaxed in the solution that we propose in the section that follows.

4. Discovering Evolving Clusters online

In this section, we present an algorithm, called *EvolvingClusters*, in order to address the specifications set in Section 3. This algorithm is fully modular, since it can discover various patterns simultaneously (in our case, *MCs* and *MCSs*), runs online, and makes no assumption about the sampling rate of moving objects (the latter is discussed in Section 4.2).

4.1. The EvolvingClusters algorithm

Algorithm 1 presents *EvolvingClusters*, which is the framework of our solution to the problem discussed in Section 3. In particular, it discovers Evolving Clusters within a trajectory data stream D, where the moving objects' locations arrive at predefined fixed timeslices (e.g. every 60 s); we call this interval the 'step' of the algorithm. (As already mentioned, the fixed and aligned sampling rate assumption will be relaxed later in Section 4.2.)

In the following paragraphs, we provide a thorough explanation regarding the functionality of the proposed algorithm. At each timeslice, Algorithm 1 calls Algorithm 2 which is

responsible for detecting MCSs and MCs in the timeslice's connectivity graph (and implements the 'spatial closeness' discussed in Definition 1), and evaluates if the available data are eligible to be used as input for Algorithm 3 which, in turn, is in charge of discovering/maintaining the evolving clusters, given the current timeslice's MCSs and MCs and the recent (active) pattern history. If Algorithm 3 returns non-empty patterns ($ActivePatterns \neq \emptyset$), Algorithm 1 outputs the ones that satisfy the given thresholds. In the opposite case (i.e. $ActivePatterns = \emptyset$) Algorithm 1, on the one hand, moves the newly discovered evolving clusters to the *ActivePatterns* set and, on the other hand, moves the previously discovered evolving clusters that were considered so far active but are not still alive to *ClosedPatterns*.

Algorithm 1: EVOLVINGCLUSTERS. Discovers Evolving Clusters online.

Input: Data stream $D = \{T_1, T_2, \ldots, T_n\}$ of Timeslices T_i consisting of objects' time-stamped locations (p_j, T_i); Distance threshold θ; Temporal threshold t; Cardinality threshold c

Output: Set *ActivePatterns* of Evolving Clusters (discovered at each timeslice)

$ActivePatterns, ClosedPatterns \leftarrow \emptyset$
for *Timeslice* T ***in*** D **do**
  $C_{MC}, C_{MCS} \leftarrow GeospatialClustering(T, \theta, c)$
  for *CurrentClusters* ***in*** $\{C_{MC}, C_{MCS}\}$ **do**
    if $ActivePatterns == \emptyset$ **then**
      $ActivePatterns \leftarrow CurrentClusters$
    else if $CurrentClusters == \emptyset$ **then**
      $ClosedPatterns \leftarrow \{ActivePattern \in ActivePatterns : ActivePattern.end - ActivePattern.start \geq t\}$
      $ActivePatterns \leftarrow \emptyset$
    else
      $ActivePatterns, InactivePatterns \leftarrow FindPatterns(CurrentClusters, ActivePatterns, c)$
      $ClosedPatterns \leftarrow \{InactivePattern \in InactivePatterns : InactivePattern.end - InactivePattern.start \geq t\}$
    end
    output $\{Pattern \in ActivePatterns : Pattern.end - Pattern.start \geq t\}$
  end
end

Algorithm 2: GEOSPATIAL CLUSTERING. Clusters coordinate points given a timeslice

Input: Timeslice $T = \{p_1, p_2, \ldots, p_n\}$ of coordinate points; Distance threshold θ; Cardinality threshold c

Output: Clusters of the timeslice's data points C_{MC}, C_{MCS}

$AdjacencyMatrix \leftarrow CreateAndFilterAdjacencyMatrix(T, \theta)$
$G \leftarrow$ Graph($edges = AdjacencyMatrix$)
$C_{MCS} \leftarrow \{C \in G.MaximalConnectedSubgraphs() : |C| \geq c\}$
$C_{MC} \leftarrow \{C \in findMCinMCS(G, C_{MCS}) : |C| \geq c\}$
return C_{MC}, C_{MCS}

Algorithm 3: FINDPATTERNS. Compares the current with the closed clusters in order to determine their evolution

Input: Clusters found at timeslice T_i *CurrentClusters*; Recent clusters history *ActivePatterns*; Cardinality threshold *c*

Output: Updated recent clusters history *ActivePatterns*′; Inactive patterns set *InactivePatterns*

1 $ActivePatterns' \leftarrow []$
2 for Pattern P_R ***in*** *CurrentClusters* **do**
3 $IntersectionSet \leftarrow []$
4 **for** Pattern P_L ***in*** *ActivePatterns* **do**
5 **if** $|P_R \cap P_L| \geq c$ **then**
6 $IntersectionSet.append(\{P_R \cap P_L, P_L.start, P_R.end\})$
7 **end**
8 $ActivePatterns'.extend(IntersectionSet)$
9 **if** $P_R \notin IntersectionSet$ **then**
10 $ActivePatterns'.append(P_R)$
11 end
12 $InactivePatterns \leftarrow \{pattern \in ActivePatterns : pattern \notin ActivePatterns'\}$
13 for Pattern P_{active} ***in*** *ActivePatterns*′ **do**
14 $DuplicatePatterns \leftarrow [pattern \in ActivePatterns' : (pattern = P_{active}), pattern \neq P_{active}]$
15 **if** $|DuplicatePatterns| \neq 0$ **then**
16 $P_{active}.start \leftarrow min(DuplicatePatterns.start)$
17 $P_{active}.end \leftarrow max(DuplicatePatterns.end)$
18 $ActivePatterns' \leftarrow ActivePatterns' - DuplicatePatterns$
19 end
20 return *ActivePatterns*′, *InactivePatterns*

Let us discuss Algorithm 2 and Algorithm 3 in more detail. Algorithm 2 evaluates if the available data are eligible to be used as input for Algorithm 3 given a timeslice, and the distance and cardinality thresholds. After detecting MCSs and in order to reduce the algorithm's search field, we detect MCs from each MCS, since (from graph theory) a clique is a subset of vertices of an undirected connected graph.

Algorithm 3 takes all possible *cases* into consideration regarding the members of pattern C_{t_i} at timeslice t_i and $C_{t_{i-1}}$ at timeslice t_{i-1}:

(1) The two patterns are identical ($C_{t_i} = C_{t_{i-1}}$)
(2) The two patterns have no common objects ($C_{t_i} \cap C_{t_{i-1}} = \emptyset$)
(3) The current pattern is a subset of the previous one ($C_{t_i} \subset C_{t_{i-1}}$)
(4) The current pattern is a superset of the previous one ($C_{t_{i-1}} \subset C_{t_i}$)
(5) The two patterns contain only some common objects ($C_{t_i} \cap C_{t_{i-1}} \neq \emptyset$, $C_{t_i} \cap C_{t_{i-1}} \subset C_{t_i}, C_{t_{i-1}}$)

Therefore, the algorithm operates as follows:

- For every pair of consecutive (with respect to time) patterns, if the cardinality of their intersection is higher than *c*, it is added to the *ActivePatterns*′ set (lines: 4–8).

- For every pattern in C_{t_i}, if the list of its intersections with all of the patterns in $C_{t_{i-1}}$ does not contain the pattern, it is added to the *ActivePatterns'* set as a new pattern (lines: 9–10).
- For every pattern in $C_{t_{i-1}}$, if it is not part of the *ActivePatterns'* set, it is added to the *InactivePatterns* set (line: 12).
- Each group of duplicate patterns in the *ActivePatterns'* set is replaced by a single record of each pattern and its starting and ending timestamps are substituted with the oldest starting and newest ending timestamps available in the duplicate group (lines: 13–19).

We observe that in all *cases* the pattern that ought to be maintained through time is the intersection of C_{t_i} and $C_{t_{i-1}}$. *Cases 2 and 3* require some extra attention. Regarding *case 2*, the intersection is an empty set; as a result, C_{t_i} should be maintained and added to the *ActivePatterns'* set. *Case 3* dictates that both the new superset and the previous pattern should be maintained since they both exist at the same time as part of C_{t_i}.

Let us explain our algorithm through the example illustrated in Figure 1. The proposed algorithm (for $c = 3$ and $t = 2$) would discover the patterns $C_1 = \{a, b, c, d, e, f, g, h, i\}$, $C_2 = \{a, b, c\}$, $C_3 = \{b, c, d, e\}$, $C_4 = \{g, h, i\}$, $C_5 = \{a, b, c, d, e\}$ and $C_6 = \{g, h, f, i\}$ as follows (per timestamp since the algorithm works in online mode):

- T_1: MCS $\langle C_1, T_1, T_1, 2\rangle$ and MCs $\langle C_2, T_1, T_1, 1\rangle$, $\langle C_3, T_1, T_1, 1\rangle$ and $\langle C_4, T_1, T_1, 1\rangle$ are discovered; Output: $\emptyset$ (since threshold t is not satified yet);
- T_2: MCS $\langle C_1, T_1, T_2, 2\rangle$ and MCs $\langle C_2, T_1, T_2, 1\rangle$, $\langle C_3, T_1, T_2, 1\rangle$ and $\langle C_4, T_1, T_2, 1\rangle$ are discovered; Output: $\{C_1, C_2, C_3, C_4\}$;
- T_3: MCS $\langle C_5, T_1, T_3, 2\rangle$ and MCs $\langle C_2, T_1, T_3, 1\rangle$, $\langle C_3, T_1, T_3, 1\rangle$ and $\langle C_4, T_1, T_3, 1\rangle$ are discovered; Output: $\{C_2, C_3, C_4, C_5\}$;
- T_4: MCS $\langle C_5, T_1, T_4, 2\rangle$ and MCs $\langle C_2, T_1, T_4, 1\rangle$, $\langle C_3, T_1, T_4, 1\rangle$, $\langle C_4, T_1, T_4, 1\rangle$ and $\langle C_6, T_4, T_4, 1\rangle$ are discovered; Output: $\{C_2, C_3, C_4, C_5\}$;
- T_5: MCS $\langle C_5, T_1, T_5, 2\rangle$ and MCs $\langle C_2, T_1, T_5, 1\rangle$, $\langle C_4, T_1, T_5, 1\rangle$ and $\langle C_6, T_4, T_5, 1\rangle$ are discovered; Output: $\{C_2, C_4, C_5, C_6\}$.

According to the above example, for timeslices T_1 to T_2, clusters C_1, C_2, C_3 and C_4 are discovered and maintained as active. At timeslice T_3, all previous clusters are maintained, except C_1 that shrinks into cluster C_5. As C_5 is present during the previous timeslices as *subset* of C_1, its starting timestamp is equal to the one of its superset (i.e. C_1). At timeslice T_4, all clusters of the previous timeslice are kept active, also a new cluster C_6 is discovered. In this peculiar case, we have to pay attention to clusters C_4 and C_6; the former is a subset of the latter, meaning that C_4 will still be maintained as an active cluster whereas C_6 will have its starting timestamp equal to the timestamp it was first discovered (i.e. T_4). Finally, at T_5, all active clusters are maintained except C_3 that cannot be found (neither as subset nor as superset) and thus, it is labeled as inactive.

4.2. Time alignment variations

Let us now relax the assumption we made in Section 4.1 about the aligned and fixed sampling rate of transmitted locations. It is clear from the input of Algorithm 1, that in order to discover Evolving Clusters the data points must be aligned on temporal axis. However, this assumption is unrealistic in many real-world applications, e.g. vessels and aircrafts broadcast their positions at an irregular sampling rate relevant to their speed,

neighborhood, etc. In order to overcome this issue of Algorithm 1, we investigate two well-known alignment methods; when Algorithm 1 runs for timeslice T_i, the location at each moving object at T_i is calculated as follows:

(1) the latest position recorded before T_i is *extrapolated* to T_i; or
(2) the first position recorded after T_i is *interpolated* at T_i.

In other words, extrapolation (interpolation)[1] maps moving object locations at T_i using preceding (succeeding, respectively) positions.

The two approaches have their pros and cons: extrapolation has virtually zero delay but is expected to yield less accurate location predictions as the distance to the last known position increases. On the other hand, interpolation deals with this problem, yielding more accurate location predictions, but at the cost of introducing a response delay (lazy update processing).

Figure 2 illustrates the above methods by presenting the temporal alignment for two moving objects O_1 and O_2. Using extrapolation, we can clearly see that we have output no matter what is the state of the objects' real points. That comes, however, at a cost because the alignment errors are accumulated over time and possibly lead us to false-positive results. Interpolation aims to alleviate these problems by 'waiting' to receive the necessary information in order to be able to interpolate. In this work, the delay is up to the next timeslice, i.e., at T_{i+1} we run Algorithm 1 for T_i; if no location is reported between T_i and T_{i+1} then the interpolation is impossible and the object is considered 'invisible' at T_i. Hence, as expected, when there are large temporal gaps between two successive data points, the output of *EvolvingClusters* will be shrinked when it runs in interpolation mode.

4.3. A note on time complexity

The theoretical complexity of the proposed *EvolvingClusters* algorithm is overwhelmed by the most 'expensive' steps, those of MCS and MC detection in the connectivity graph (Algorithm 2, lines 4–5). Detecting *MCSs* can be reduced to a simple depth-first search and therefore its complexity is $\mathcal{O}(|V| + |E|)$, where V, E are the graph's vertices and edges, respectively. In our case, $|V|$ is the number of objects having at least another object in their neighborhood (defined by distance threshold θ) and $|E|$ is the number of pairs of objects with distance less than or equal to θ. On the other hand, according to Bron and

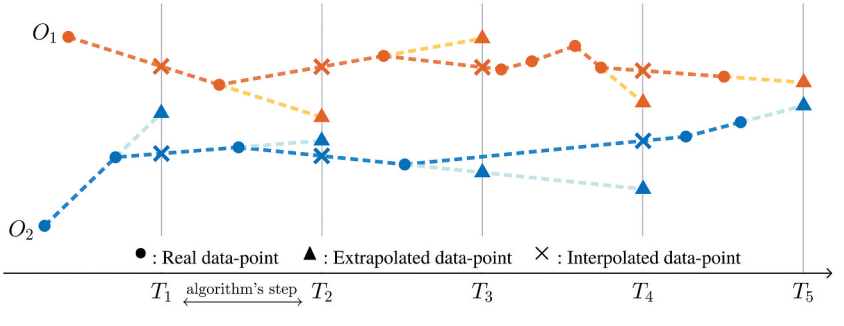


Figure 2. Online temporal alignment example for two moving objects; dots correspond to actual positions whereas triangles and stars correspond to extrapolated and interpolated, respectively, positions at timeslices $T_i, i \in 1, \ldots, 5$.

Kerbosch (1973), extracting all maximal *Cliques* from a graph has a complexity of $\mathcal{O}(3^{|V|/3})$. In other words, the worst-case complexity of the method is exponential with respect to the number of nodes within the graph (i.e. at the worst case, the number of concurrent objects within a timeslice).

Nevertheless, according to our experiments, our method is able to run in online mode in all real-world situations we have experimented with. In detail, focusing on the most crowded day in all four datasets, and running Algorithm 1 with a step of *60 sec*. (for Brest, GeoLife and FlightAware) and *30 sec*. (for Porto; the datasets used in our experimental study are presented in Section 5.1), Figure 3 illustrates the (maximum recorded) response time for the most 'expensive' stage of Algorithm 1, which is the adjacency matrix computation.

As it appears in the chart, the response time stays below 70 ms for thousands of concurrent objects and increases up to (approx.) 216 ms for 2,360 objects, which corresponds to the most crowded timeslice in our experimental study. While in theory, the discovery of MCS/MC is the most 'expensive' stage compared to the computation of the adjacency matrix, in real-life it turns out to be quite the opposite. In general, in our experimental study, the time elapsed for discovering patterns (either MCS or MC) in the connectivity graph (i.e. the filtered adjacency matrix) never exceeded 2.5 ms (therefore, it is not illustrated in Figure 3).

Typically, the created connectivity graphs include fewer objects than those that are concurrently alive at the timeslice at hand and this is, of course, due to the distance threshold θ. At this point, we should also mention that as we increased θ (in the range of values discussed in the experimental section) we did not observe any significant change in the elapsed time for each of the algorithm's steps.

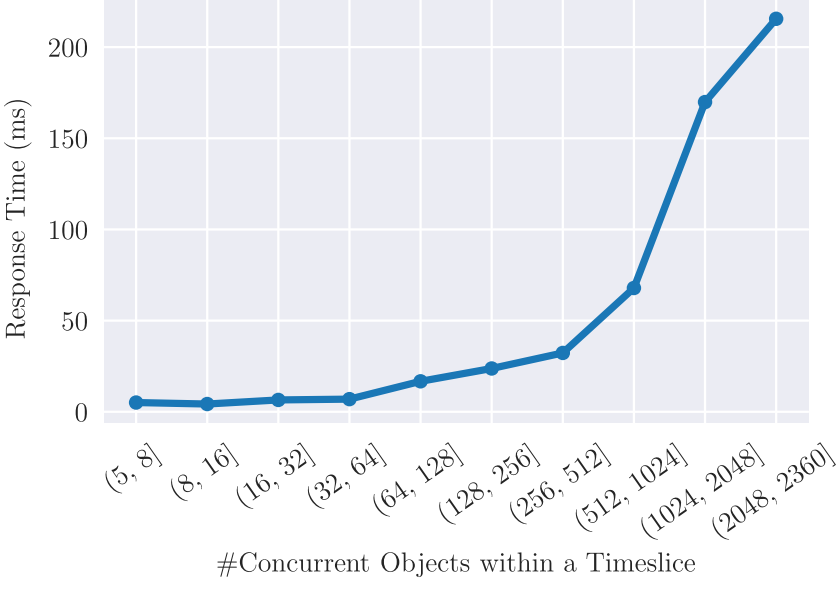


Figure 3. Maximum adjacency matrix computation time vs. #concurrent objects within a timeslice.

Furthermore, in order to accelerate the 'Adjacency Matrix Computation' stage, we used the Ball Tree[2] algorithm, using off-the-shelf toolkits[3] tuned using their default parameters, and the 'haversine' distance metric. Last but not least, it is obvious that the performance of Algorithm 1 is correlated to the hardware of the experimental setup (c.f. Section 5.2).

4.4. A note on the generality of EvolvingClusters

As briefly mentioned in Section 1, the reason why our proposed method is based on graphs is two-fold: on the one hand, although we currently discuss two interesting cases extracted from a graph, namely MCs and MCSs, this does not prevent us from extending our work in the future with other pattern types from graph theory (minimum spanning trees, utility graphs, etc.) that may appear to be interesting in mobility applications. On the other hand, by establishing our approach in a sound theoretical base, as that of graphs, we are able to be quite general and consider existing co-movement patterns, such as flocks and convoys, which are the most popular in the family, as instantiations of our evolving clusters.

In order to support the latter reason, in the following paragraphs, we prove that an Evolving Cluster of type MC (MCS) simulates flocks (convoys, respectively). In our opinion, this is a result that significantly differentiates our work from related work discussed in Section 2.

Proposition 1. *Consider a set* $S = \{p_1, p_2, \ldots, p_n\}$ *of n points in 2D plane, with all their pairwise distances upper bounded by* θ, *i.e.,* $i, j \in 1, n, i \neq j$, *it holds that* $d(p_i, p_j) \leq \theta$. *It is shown that S is geometrically enclosed in a circle with radius less than* θ.

Proof. Straightforward, from Jung's theorem (Weisstein n.d.).

Corollary 1. *From Proposition 1, it turns out that an Evolving Cluster of type MC with distance threshold* θ *corresponds to a flock with distance threshold (circle radius) less than* θ, *as it is defined in* Gudmundsson and van Kreveld (2006), Benkert *et al*. (2008).

Proposition 2. *Consider a set* $S = \{p_1, p_2, \ldots, p_n\}$ *of n points in 2D plane where the following two conditions hold: (i) for each point in S there is another point in S with distance being less than or equal to* θ, *i.e.,* $i \in 1, n, \exists j \in 1, n : d(p_i, p_j) \leq \theta, i \neq j$, *and (ii) there is a path between any pair of points in S where the distance from point to point in the path is less than or equal to* θ, *i.e.,* $i, j \in 1, n, i \neq j, \exists \langle p_{start}, p_k, p_{k'}, \ldots, p_{k''}, p_{end} \rangle$, *where* $p_{start} = p_i$, $p_{end} = p_j$, $d(p_{start}, p_k) \leq \theta, d(p_k, p_{k'}) \leq \theta, \ldots, d(p_{k''}, p_{end}) \leq \theta$. *It is shown that S is a density-connected set of points with distance threshold* θ, *according to the DBSCAN paradigm* (Ester *et al*. 1996).

Proof. Straightforward, from the conditions of the proposition.

Corollary 2. *From Proposition 2, it turns out that an Evolving Cluster of type MCS with distance threshold* θ *corresponds to a convoy with distance threshold* θ, *as it is defined in* Jeung *et al*. (2008a, 2008b).

The experimental validation of these theoretical results appears in Section 5.2

5. Experimental study

In this section, we evaluate our method on three different mobility domains, namely maritime, urban and aviation.

5.1. Datasets & preprocessing

It has been established that sensor-based information is sensitive to errors due to device malfunctioning. Therefore, a necessary step before performing any data analytics task is that of pre-processing. In general, pre-processing of GPS-based location data includes data cleansing (e.g. noise elimination, location smoothing, etc.) as well as data transformation (e.g. segmentation, semantic enrichment, etc.), tasks which are necessary for whatever analysis is going to follow (fixed rate resampling, trajectory segmentation, etc.) (Pelekis and Theodoridis 2014).

5.1.1. Maritime domain

In the maritime domain, where port locations are usually given as point geometries in open datasets, a popular approach for trajectory segmentation is that of considering a circle of radius ρ around each port's location in order to approximate their geometry and then, detect port entry and exit points for each vessel trajectory (Spatial-based Segmentation). Then, for each trajectory, we may detect pairs of points with temporal difference greater than a given threshold dt (Temporal-based Segmentation). These pairs signify the transition from the current to the next *Trip*. The aforementioned methodology is illustrated in Figure 4(a), where the raw location information is compared with a port's location, thus the vessel's movement is segmented into four trajectories/trips.

From the maritime domain, we use the popular open 'Brest' dataset[4] (Ray *et al*. 2019); it contains AIS information regarding maritime movement (mainly) in Brest Bay, France. The (raw) dataset ranges in time and space as follows:

- Temporal range: October 1st, 2015 – March 31st, 2016 (6 months);
- Spatial range: longitude in [−10.0, 0.0], latitude in [45.0, 51.0].

A map visualization of (a part of) the dataset is illustrated in Figure 4(b). The dataset consists of multiple data types, but for the purposes of our study, we exploit the entire vessel-dynamic (i.e. related to the vessels' movement) information, while from the geo-related information we are only interested in the ports' locations, information vital for the analysis we perform. The vessel-dynamic dataset consists of approximately 19 million records from 5,055 vessels. During the preprocessing stage, we set the following thresholds: $speed_{max} = 50$ knots, $\rho = 2$ km, $dt = 1$ hour.

The rationale behind these thresholds is empirical and stems from the characteristics of the dataset. After calculating the point-wise velocity for the locations of our dataset, its area chart shows a rapid decrease at around 40 knots. In order to be well-within the margin of error, while taking into account that the maximum recorded velocity for commercial vessels/fisheries is around 50 knots, we decided to set $speed_{max}$ to 50 knots. Moreover, ρ was empirically set in an attempt to cover as much of the ports real area as possible while not exceeding their horizon beyond the open sea. Finally, the choice of dt was arbitrarily made in the sense of working shifts.

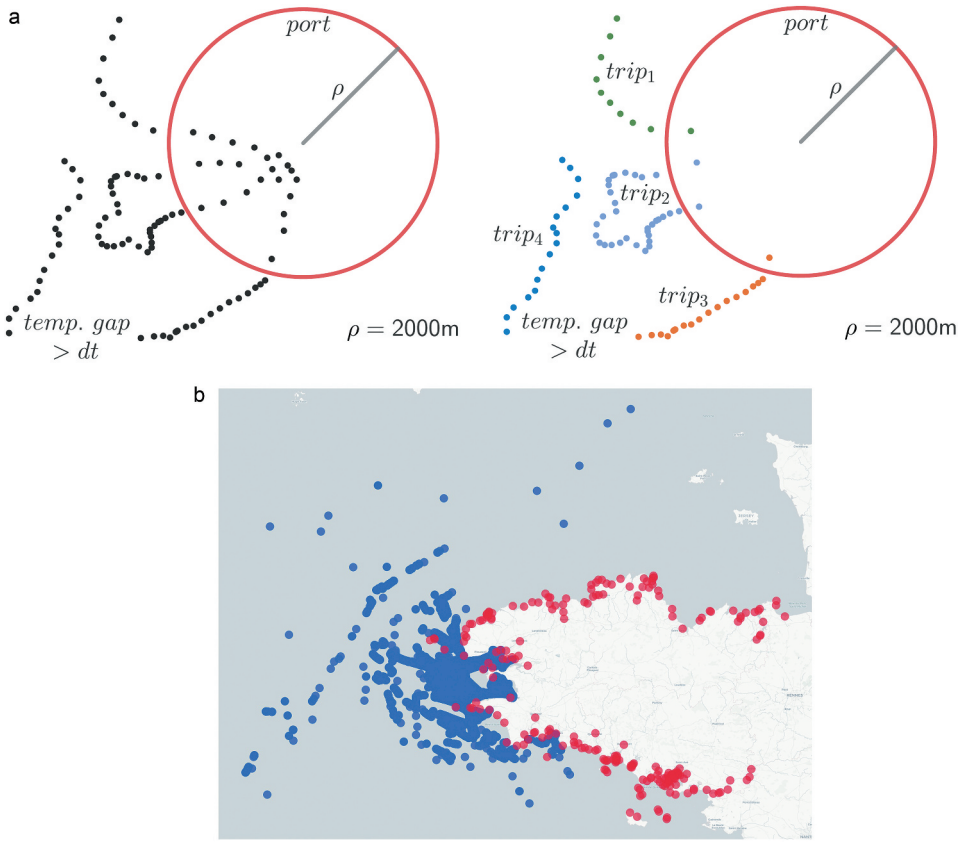


Figure 4. (a) A sample vessel trajectory before (left) and after its segmentation into trips (right); (b) A snapshot from the Brest dataset – sample of GPS locations (blue dots) and Ports (red dots) on March 1st, 2016.

5.1.2. Aviation domain

As in the previous case, in the aviation domain, a popular approach for trajectory segmentation is to create a circle of radius ρ around each airport's location in order to approximate their geometry and then, detect airport entry/exit points for each aircraft trajectory. Since the aircrafts are moving in 3D space, in order to exclude false-positive entry points, we also set a height threshold h for each airport's location, which approximates the aircraft's intention for landing/takeoff.

The above discussion is illustrated in Figure 5(a). Notice that as $trip_1$ enters the circle twice, it is segmented only at the second time, where it eventually lands. The first 'entrance' is the case where the aircraft's altitude is significantly higher than that of the airport's.

From the aviation domain, we use FlightAware ADS-B[5] dataset (Patroumpas *et al*. 2018) provided by FlightAware©, derived from ADS-B messages in Europe. The dataset ranges in time and space as follows:

- Temporal range: March 10th, 2017 (24 hours);
- Spatial range: longitude in [–31.26, 39.87], latitude in [27.64, 72.56], altitude in [0, 64400].

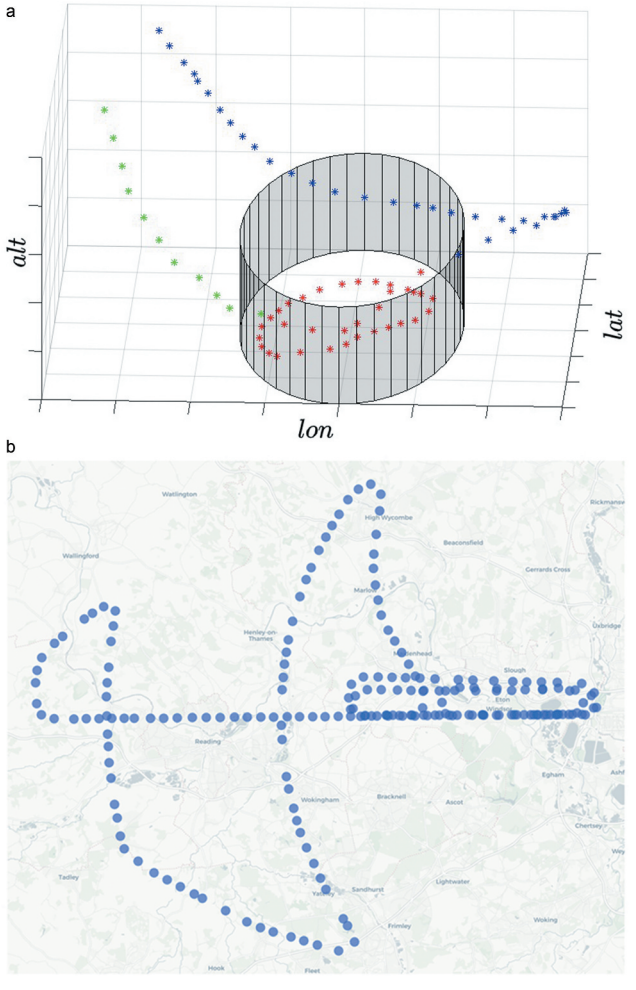


Figure 5. (a) A sample aircraft trajectory after its segmentation into trips. (3D View); (b) A snapshot from the FlightAware dataset – sample of an aircraft's ADS-B messages (blue dots) on March 10th, 2017 from 00:00 to 01:00.

A map visualization of (a part of) the dataset is illustrated in Figure 5(b). The original dataset contains information related to the aircrafts' movement. In particular, it consists of approximately 3.7 million records from 24,416 aircrafts. During the preprocessing stage, we set the following thresholds: $speed_{max} = 600$ knots, $\rho = 2$ km ($\times$ 3000 ft. height), $dt = 1$ hour.

Below we present the rationale behind these thresholds. After calculating the pointwise velocity for the locations of our dataset, its area chart shows a rapid decrease at around 400 knots. In order to be well-within the margin of error, while taking into account that the maximum recorded velocity for commercial vessels/fisheries is around 600 knots, we decided to set $speed_{max}$ to 600 knots. Furthermore, the airports' radius (ρ) was set arbitrarily, in order to cover as much of their respective real area as possible. Their height (h) however, was set in order to account for the altitude threshold in which an aircraft is

considered to be in landing/takeoff phase, according to Air Traffic Control (ATC) stipulations. Finally, the choice of *dt* was arbitrarily made in the sense of flight shifts.

5.1.3. Urban domain

From the urban domain, we use two popular datasets in the field of mobility data analytics, the GeoLife[6] dataset (Zheng *et al*. 2008, 2009, 2010), which contains information regarding urban movement (mainly) in Beijing, China and the Porto[7] dataset, which contains information regarding taxi movement (mainly) in Porto, Portugal.

The GeoLife dataset ranges in time and space as follows:

- Temporal range: April 12th, 2007 – July 27th, 2012 (approx. 5 years);
- Spatial range: longitude in [73.50, 134.78], latitude in [17.96, 53.56] (restricted to China).

The Porto dataset ranges in time and space as follows:

- Temporal range: July 1st, 2013 – June 30th, 2014 (1 year);
- Spatial range: longitude in [−8.79, −7.88], latitude in [41.00, 41.47] (restricted to Portugal).

A map visualization of (a part of) both datasets is illustrated in Figure 6. The GeoLife dataset contains information related to the users' movement. In particular, it consists of approximately 23.5 million records from 181 users organized in 18,454 trips, with some of their trips being semantically annotated with respect to the means of transportation they used at that time. The Porto dataset contains information related to the taxis' movement. In particular, it consists of approximately 83.4 million records from 444 taxis organized in 1,704,685 trips.

A necessary clarification at this point is that in both datasets we omit the preprocessing step and we accept them 'as-is'. The main reason for this, is due to the fact that the raw points are already organized into trips, and – in the case of the GeoLife dataset – are neatly labeled with respect to the vehicle used. That being said, the only steps prior to our experimental study are, for the GeoLife dataset, to restrict its spatial range to the China region, and for the Porto dataset, to remove from the original dataset the trajectories that appear to have missing points.

5.1.4. Dataset statistics

Table 1 demonstrates a few statistics about the datasets (after the preprocessing stage, where it was performed). Also, Figure 7 illustrates the temporal density (i.e. sampling rate) of the objects' trips for each of the experimental datasets (except the Porto dataset, which is omitted because the trips of Porto Dataset have a constant sampling rate of 15 s). Observing the aforementioned figure, we conclude that the vast majority of objects' locations are transmitted in seconds.

5.2. Experimental setup and results

Having preprocessed our datasets using the methodology and thresholds presented in Section 5.1, we experimentally evaluated our EvolvingClusters method on all four datasets

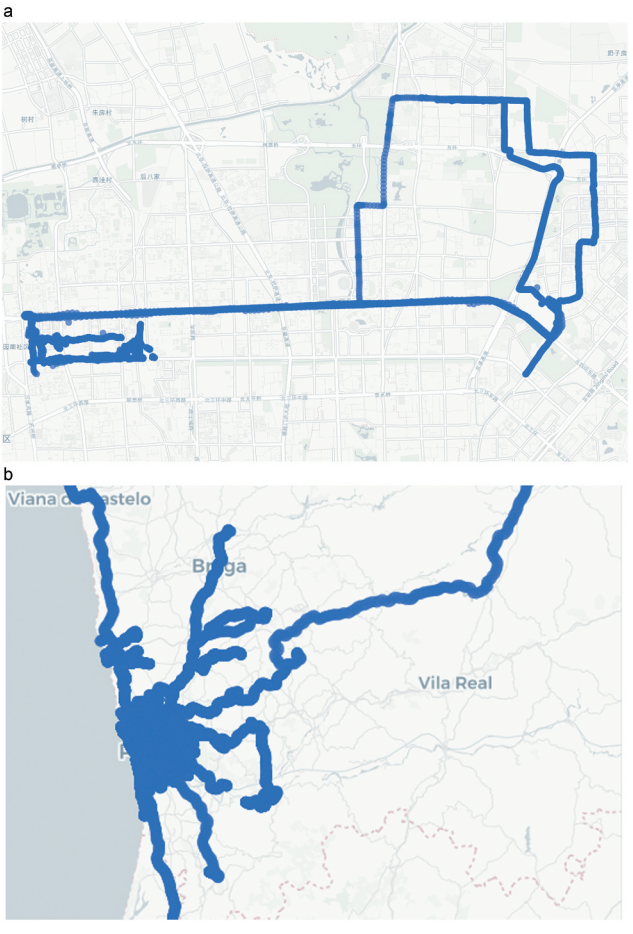


Figure 6. Snapshots from (a) the Geolife; and (b) the Porto datasets – sample of GPS positions (blue dots) on October 20–22th, 2011 and June 10th, 2014, respectively.

Table 1. Statistics of the datasets used in our experimental study.

	Brest	FlightAware	GeoLife	Porto
#data points	9,546,860	3,611,892	23,590,432	83,408,417
#objects	3,117	23,532	181	444
#trips	28,510	26,103	18,454	1,704,685
#data points per object (min.; avg.; max.)	3; 3,063; 1,100,346	3; 153; 1,345	17; 130,334; 1,980,458	1; 187,857; 475,391
#trips per object (min.; avg.; max.)	1; 9; 869	1; 1; 18	1; 102; 2,139	1; 3,839; 10,497

on a wide range of values for each parameter and dataset, as Table 2 illustrates. In particular, for FlightAware we use an increased distance threshold following Air Traffic Control (ATC) guidelines[8] whereas for Porto and GeoLife we set, on the one hand, the distance threshold θ by accounting the average width of a road and the spatial error[9] of the GPS receiver and, on the other hand, the temporal threshold t by taking into account the trips' average duration.

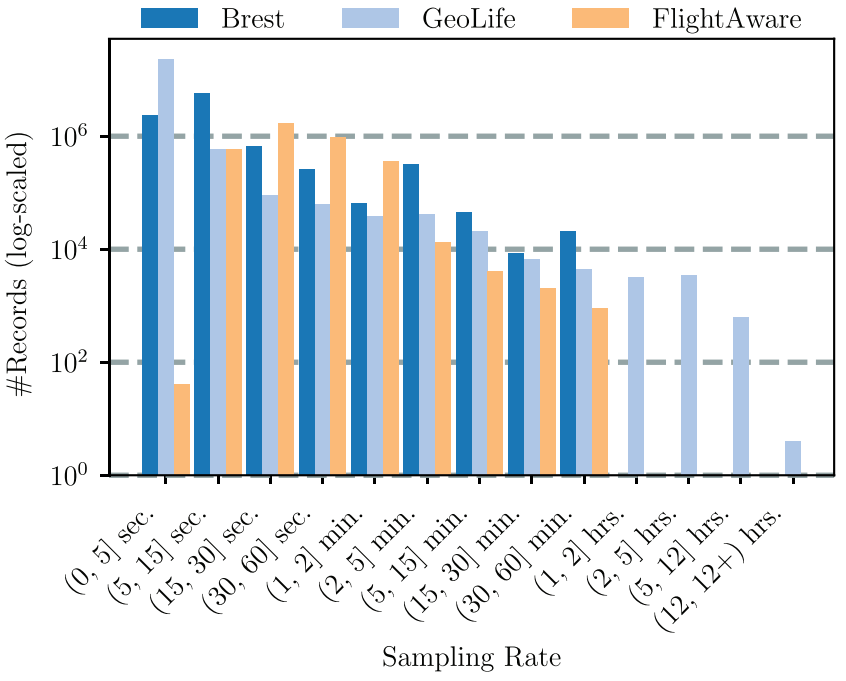


Figure 7. Sampling rate distributions of trips for each of the three datasets.

The objectives of our experimental study are related to:

- parameter (c, t, θ) sensitivity;
- the effect of the algorithm's step;
- the accuracy of online temporal alignment variations (extrapolate vs. interpolate); and
- the quality of the discovered patterns with respect to the seminal Convoys and Flocks.

All algorithms were implemented in Python3 (via Anaconda3[10] virtual environments). The experiments were conducted using Apache Kafka® with 1 topic for the transmitted locations – loaded from a CSV file – and 1 consumer for discovering evolving clusters. The machine we used is a single node with 8 CPU cores, 16 GB of RAM and 256 GB of HDD, provided by okeanos-knossos,[11] an IAAS service for the Greek Research and Academic Community.

5.2.1. Parameter sensitivity

In this section, we discuss the sensitivity of our algorithm with respect to its thresholds. The metrics that we use focus on the algorithm's qualitative results, i.e., the number of evolving clusters produced, their average distance travelled as well as their average duration.

Figure 8 summarizes the previously discussed experiment over the GeoLife dataset. We observe that as θ increases, the overall (i.e. average among all three datasets) number of discovered evolving clusters increases, while the average duration and distance travelled continuously decreases for both MCSs and MCs, with some slight

Table 2. Datasets' threshold ranges for evolving cluster discovery (default values are in bold and underlined).

	Brest	FlightAware	GeoLife	Porto
Cardinality (c)	**5**, 10, 15	**5**, 10, 15	**5**, 10, 15	**5**, 10, 15
Temporal (t)	**15**, 30, 45	5, **10**, 15	**10**, 15, 20	**1**, 2, 5
Distance (θ)	500, **1000**, 1500	2000, **3000**, 4000	10, 20, **30**	10, 20, **30**

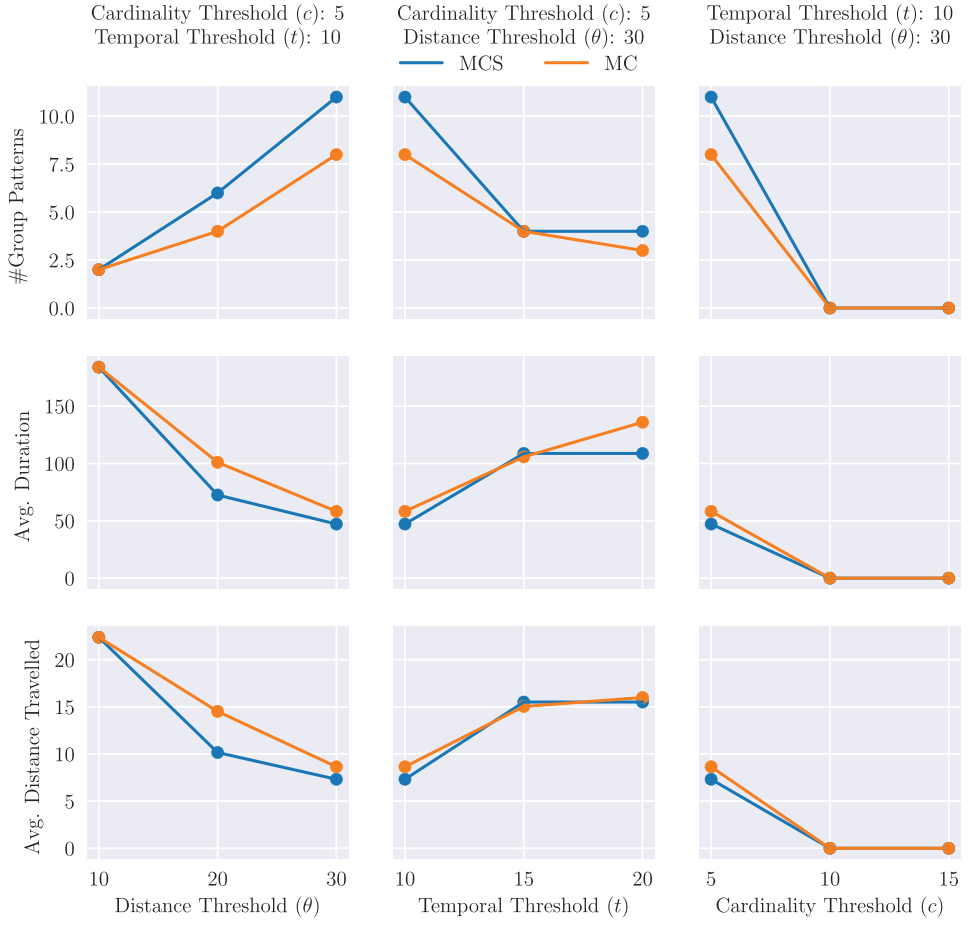


Figure 8. Statistics on discovered evolving clusters over the *GeoLife* dataset: Cardinality (c), Distance (θ) and Temporal Threshold (t) vs. #Group Patterns; Avg. Duration (minutes); and Avg. Distance Travelled (km).

fluctuations for the former, as a result of increasing the reachability threshold. Moreover, as t increases, both discovered MCSs and MCs show a gradual increase at their average duration and distance travelled, but at the cost of getting fewer clusters. Finally, as c increases, both cluster types show a steep decrease in their overall cardinality as well as in their average duration and distance travelled. Similar behaviour was presented in our experimentation with the other datasets as well, therefore we omit the presentation of their respective charts.

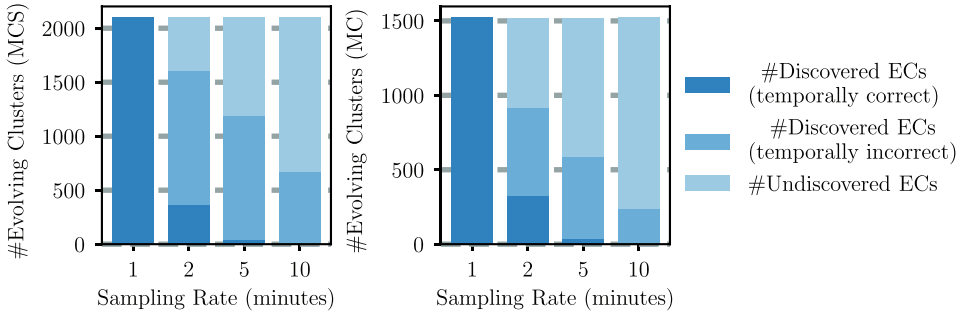


Figure 9. The effect of the algorithm's step over the *Brest* dataset – Quantitative comparison on discovered patterns with respect to the step.

5.2.2. The effect of the algorithm's step

The second set of experiments that we conduct on EvolvingClusters is the quality of its results with respect to the algorithm's step (i.e. the frequency of the incoming timeslices – c.f. Figure 2). We assessed the quality of the output by comparing it against 'ground truth', which is defined as the resulted timeslices when using offline linear interpolation at a rate of 1 min.

According to Figure 9, as the step increases (i.e. the temporal density becomes more sparse) so does the quality loss. In this figure, '#Discovered ECs (temporally correct)' corresponds to the number of discovered evolving clusters, which were identical to ones that were discovered in the ground truth. The label '#Discovered ECs (temporally incorrect)' corresponds to the number of discovered evolving clusters, which were almost identical to ones that were discovered in the ground truth, with the single difference that their respective starting and/or the ending times (slightly) deviate. Finally, the label '#Undiscovered ECs' corresponds to the number of evolving clusters that were not discovered at all, according to the ground truth.

More specifically, focusing on the Brest dataset and assuming as control (for our comparisons) a step of 1 min., we observe that as the temporal sparsity of an incoming timeslice increases, the cardinality of the discovered evolving clusters (steadily) decreases. In addition, we observe that the discovered evolving clusters become more 'simplified', thus introducing an 'aliasing' effect in the temporal dimension, that is seemingly correlated to the selected step. For instance, according to Figure 9, a step equal to 5 min provides very limited (accurate) output – with respect to the patterns' start and ending timestamps –, which gets zeroed when the step gets equal to or higher than 15 min. (not illustrated in Figure 9), not to mention order of hours.

5.2.3. Online temporal alignment accuracy

The third set of experiments that we conduct on EvolvingClusters evaluates the time alignment variations proposed in Section 4.2. For demonstration purposes, we choose the Porto dataset because, among all four datasets (c.f. Section 5.1), it is the only one that has a stable sampling rate, equal to *15 s* per data point per trip. Figure 10, illustrates the difference among the two alignment variations.

More specifically, focusing on the discovered MCSs of the Porto Dataset, with the parameters of our algorithm set at their respective default values and setting its step at

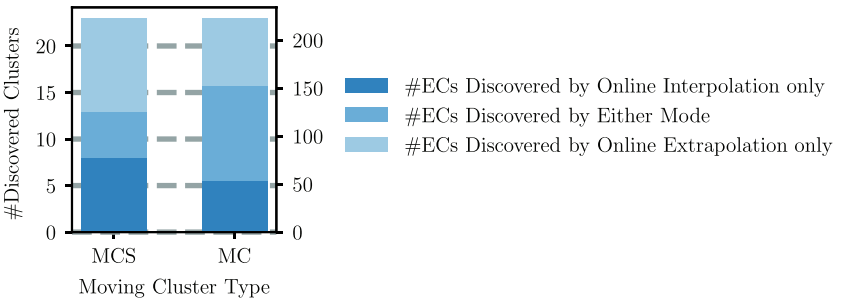


Figure 10. The effect of temporal alignment over the *Porto* dataset – Quantitative comparison on discovered patterns with respect to the alignment method.

30 sec., we discover 13 MCSs using interpolation and 15 MCSs using extrapolation; from these patterns, 5 of them were discovered in either mode. Similar observations can be made based on the discovered MCs, with the difference that extrapolation discovered slightly more patterns compared to interpolation (169 and 153, respectively, with 99 of them being discovered in either mode).

The aforementioned conclusions are correlated to the accuracy vs. speed trade-off. Using linear extrapolation we are able to get (virtually) instant, i.e., accounting only the algorithm's response time, results but with some loss with respect to the discovered patterns as well as their respective duration. On the other hand, by using linear interpolation we increase the quality of the discovered clusters, but at the cost of introducing a temporal delay, equal to the algorithm's step.

5.2.4. EvolvingClusters vs. Flocks/Convoys

Finally, the last set of experiments we conduct is on the evaluation of EvolvingClusters' output compared to that produced by Flock and Convoy implementations.

For flock pattern mining, we use the implementation of 'Moving Flock Finder',[12] while for convoy pattern mining, since we could not find proper open software, we used our method with Algorithm 2 being replaced by the popular DBSCAN (Ester *et al*. 1996) algorithm with $eps = \theta$ and $min_pts = c$, based on the problem definition given by Jeung *et al*. (2008a).

For the output comparison, we use the synthetic *Oldenburg* dataset,[13] following the guidelines of the *Moving Flock Finder* web page. After preparing the aforementioned dataset (in order to be interpretable by each pattern mining implementation), we discover convoys, flocks, MCS and MC using the following threshold values: $c = 3$, $t = 3$ min., $\theta = 150$ m. In general, we get the same results as Flocks and Convoys (albeit with some – slight – temporal aliasing), while sometimes we discover additional patterns that were not discovered by the latter implementations. More specifically, our algorithm discovered 320 MCSs, exactly the same as Convoys, with no temporal aliasing. On the other hand, *EvolvingClusters* discovered 160 MCs, while *Moving Flock Finder* discovered 218 Flocks, with only 96 of them found in the intersection set (slightly) temporal aliased.

The main reason behind the difference between the two outputs is due to the extra pruning *Moving Flock Finder* makes in its pattern discovery phase, which is beyond the original definition of the Flock pattern. According to the web page of the

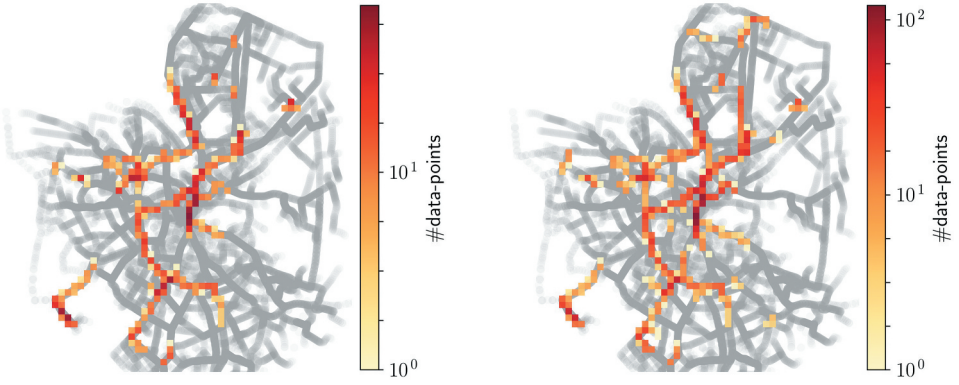


Figure 11. Heatmap of discovered Group Patterns within the Oldenburg Dataset (left: MC; right: Flocks).

implementation, *'To prune out patterns whose members solely stay in 1 location, the extent of the distance covered by the flock is computed and those with extent smaller than the radius are discarded.'*

In a more qualitative comparison between Flocks and MC, by comparing the heatmap of the objects' activity illustrated in Figure 11, we observe that the gist of the discovered Flocks is well captured, but with some minor loss over the mild-to-moderate traffic areas.

Overall, we conclude that our EvolvingClusters method is able to emulate the Flock and Convoy mobility patterns by MC and MCS pattern types, respectively.

6. Towards the exploitation of Evolving Clusters

A very important stage in the data analytics hierarchy is data storytelling; in other words, how to deduce meaningful and easy to be interpreted outcomes from a (curated) dataset at hand. After executing EvolvingClusters on all four datasets we can present some further findings, aiming to reach some – more insightful – conclusions regarding the objects' activity.

In Figures 12–14, we present the discovered evolving clusters (MCS – left; MC – right) heatmap in all three domains we used in our experimentation (maritime, urban and aviation, respectively), with the method's parameters set at their respective default values (c.f. Table 2). Due to the fact that the discovered MCs are essentially subsets of MCSs and in order to avoid repetitions, we choose to focus our discussion on the discovered MCSs' footprint (Figures 12–14; left pane).

Starting with the maritime domain (Brest dataset), if we are interested in the *fishing* vessels' activity and what their respective evolving clusters means to us, Figure 12 provides an answer: it can be observed that the fisheries form several anchorages at Dourarnenez Bay, as well as at Brest Bay. The one pattern which is located at the far left (at the open sea) means that several fisheries are concurrently fishing, and by the respective colorbar we deduce that the fishing pressure at that region is moderate. Even further, we can observe moving patterns near Dourarnenez Bay as well as Brest Bay, indicating either high fishing activity (with a respectively high fishing pressure level) or other (probably

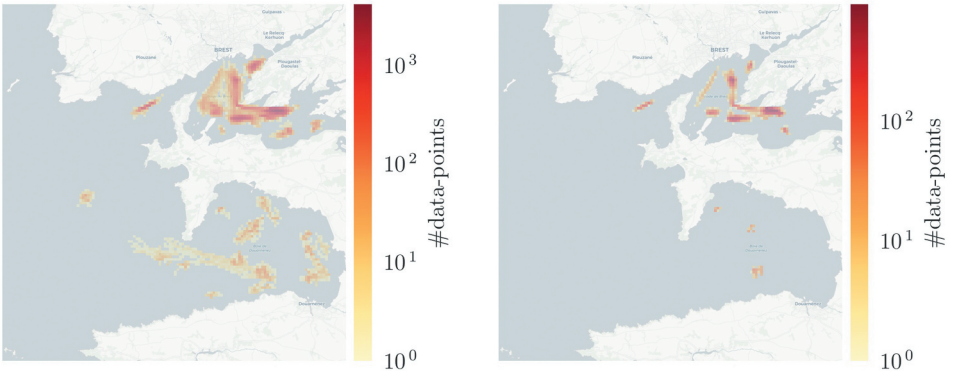


Figure 12. Heatmap of discovered Evolving Clusters in the maritime domain: Brest (left: MCS; right: MC).



Figure 13. Heatmap of discovered Evolving Clusters in the urban domain: (a) GeoLife; (b) Porto (left: MCS; right: MC).

suspicious) activities, such as transshipment and intentional AIS switch-off (Kontopoulos *et al*. 2020). These findings may trigger domain experts into further investigating these occurences and reach some meaningful conclusions.

Moving to the urban domain and regarding the GeoLife dataset, we focus on the *road* vehicles (i.e. buses, taxis, bikes and cars) as well as all data points labeled as 'Other' (due to the fact that this label contains indistinguishable information regarding the vehicles

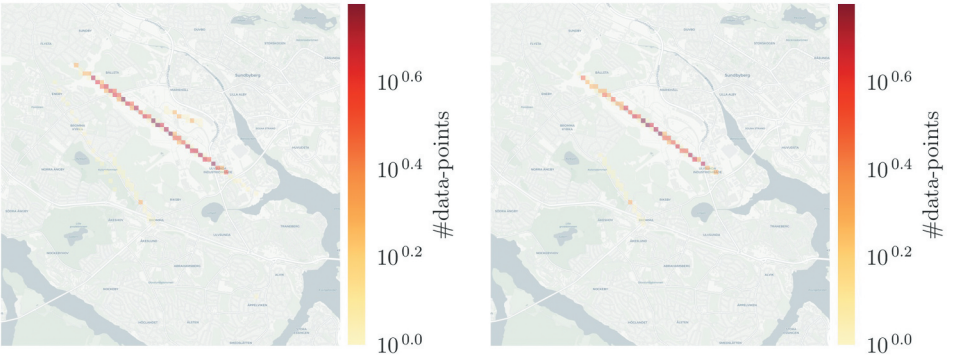


Figure 14. Heatmap of discovered Evolving Clusters in the aviation domain: FlightAware (left: MCS; right: MC).

used). As Figure 13(a) illustrates, most traffic lies around Beijing, China, with empasis on major traffic areas (such as museums and university campuses, as it can be easily deduced when comparing the spatial footprint of the discovered evolving clusters with open spatial data sources, like Wikimapia.org, LinkedGeoData.org, etc.). Combined with the hours they are created, a domain expert (e.g. transportation analyst) could easily determine where and when a rush hour will occur, thus using EvolvingClusters as a utility to effectively reroute urban traffic.

On the other hand, regarding the Porto dataset, we are interested in the movement trends of the taxis in Porto, Portugal. As Figure 13(b) illustrates, most traffic lies around the historic city center, with emphasis on major traffic areas (such as museums, sightseeing areas, restaurants and train stations) as these are the most likely pick up/drop off places. When these places are combined with the hours of the day they appear, a domain expert (e.g. a transportation analyst) could easily determine where and when spikes in traffic will occur, thus using EvolvingClusters could potentially be used as a utility to effectively disperse the taxi fleet, so as to maximise availability, and therefore, profit.

Finally, regarding the aviation domain (FlightAware dataset) and having taken the international aviation rules into consideration, our method could be exploited as a tool for Air Traffic Control (ATC) purposes. Figure 14 illustrates the previous discussion by showing the (few) evolving clusters that appear in Stockholm, Sweden. Focusing on the area of interest (i.e. the patterns' data points) we observe that the aircrafts' movement lies, as expected, around the local airport, thus the evolving clusters discovered may be considered as a false alarm, since their movement is anyway regulated by the local Control Tower.

In conclusion, the discovery of Evolving Clusters can offer us great insight regarding the objects' behaviour, in land, sea, and air transportation. With this knowledge, actions can be taken from either the perspective of the industry (e.g. fleet management) or in some cases, law enforcement (e.g. transshipment, ATC).

7. Conclusions and future work

In this paper, we proposed a unified online group pattern mining algorithm, called EvolvingClusters, which is used to discover collective movement behaviour (like flocks

and convoys) by monitoring the activity of multiple clusters through time and space. The algorithm is graph-based in the sense that it maintains evolving *Maximal Cliques (MCs)* and *Maximal Connected Subgraphs (MCSs)*, searched in a connectivity graph built upon the objects' data points, thus simulating spherical and density-based evolving clusters, respectively. The algorithm runs in online mode and differentiates from related work in that it is able to simulate two popular types of group patterns (namely, flocks and convoys), and thanks to its graph-based structure, paves the way for others that may appear to be equally interesting. Moreover, it relaxes the assumption of related work about the synchronized and fixed sampling rate, which is impractical in most real-world settings.

Our study on four representative real-world datasets from the maritime, aviation and urban domain, demonstrates the efficiency and effectiveness of the proposed algorithm. Thus, based on the potential applications, some of which were discussed in Section 6 as eventual datastories, as well as the quality of the results produced, we argue that EvolvingClusters algorithm can turn out to be a valuable tool for researchers and practitioners alike in the field of Mobility Data Analytics.

In the future, we aim to exploit some graph-oriented metrics (e.g. diameter, centrality) in order to (a) further characterize the discovered clusters; and (b) prune the redundant clusters, thus retaining only the most essential information. We also plan to study the evolution of discovered clusters with time (e.g. evolving clusters may appear/disappear, split/merge, etc.) inspired by Spiliopoulou *et al*. (2006). Another milestone for our method is to implement it in a big data programming environment in order to be able to efficiently monitor tens or hundreds of thousands of concurrent objects. As a more ambitious objective, we aim to develop a framework that will use the discovered Evolving Clusters in order to classify moving objects into different classes based on their behaviour. By extracting as much information as possible from the available data and combining a well trained classifier with a well defined set of groups with similar behaviour, we aim to create a system able to model and – if possible – predict e.g. irregular/anomalous activities in land, sea and air.

Notes

1. In this paper, we use the linear interpolation/extrapolation models. Other models, e.g. polynomial, spline, etc., can be used as well but their assessment is beyond our scope.
2. https://en.wikipedia.org/wiki/Ball_treeBall Tree, From Wikipedia the free encyclopedia.
3. https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.BallTree.html#sklearn.neighbors.BallTreeBallTree, scikit-learn Documentation.
4. The dataset is publicly available at https://zenodo.org/record/1167595#.XjSPOcYzaV5zenodo.org.
5. The dataset is publicly available at http://chorochronos.datastories.org/?q=content/flightaware-ads-b-march-2017chorochronos.datastories.org.
6. The dataset is publicly available at https://research.microsoft.com/en-us/downloads/b16d359d-d164-469e-9fd4-daa38f2b2e13/research.microsoft.com
7. The dataset is publicly available at https://www.kaggle.com/c/pkdd-15-taxi-trip-time-prediction-iikaggle.com
8. https://en.wikipedia.org/wiki/Separation_(aeronautics)Separation (aeronautics) – From Wikipedia, the free encyclopedia

9. https://en.wikipedia.org/wiki/Lane#Lane_widthLane width (urban transportation) – From Wikipedia, the free encyclopedia
10. https://www.anaconda.com/
11. https://okeanos-knossos.grnet.gr/home/
12. 'Moving Flock Finder' is publicly available at https://kdd.isti.cnr.it/moflock-moving-flock-finderkdd.isti.cnr.it
13. The dataset is publicly available at https://kdd.isti.cnr.it/moflock-moving-flock-finderkdd.isti.cnr.it

Acknowledgments

This work was partially supported by projects i4Sea (grant T1EDK-03268) and Track&Know (grant agreement No 780754), which have received funding by the European Regional Development Fund of the EU and Greek national funds (through the Operational Program Competitiveness, Entrepreneurship and Innovation, under the call Research-Create-Innovate) and the EU Horizon 2020 R&I Programme, respectively.

Data and codes availability statement

The data and codes that support the findings of this study are available at https://doi.org/10.6084/m9.figshare.11918715.

Disclosure statement

No potential conflict of interest was reported by the author(s).

References

Andrienko, G.L., *et al*., 2013. *Visual analytics of movement*. New York:Springer.
Benkert, M., *et al*. 2008. Reporting flock patterns. *Computational Geometry*, 41 (3), 111–125. doi:10.1016/j.comgeo.2007.10.003
Bron, C. and Kerbosch, J., 1973. Finding all cliques of an undirected graph (algorithm 457). *Communications of the ACM*, 16 (9), 575–576. doi:10.1145/362342.362367
Chen, L., *et al*., 2019. Real-time distributed co-movement pattern detection on streaming trajectories. *PVLDB*, 12 (10), 1208–1220.
Ester, M., *et al*., 1996. A density-based algorithm for discovering clusters in large spatial databases with noise. *In*: *KDD*. Portland, Oregon: AAAI Press, 226–231.
Fan, Q., *et al*., 2016. A general and parallel platform for mining co-movement patterns over large-scale trajectories. *PVLDB*, 10 (4), 313–324.
Fang, Z., *et al*., 2020. Coming: A real-time co-movement mining system for streaming trajectories. *In*: *SIGMOD*. New York:ACM Press, 2777–2780.
Giannotti, F. and Pedreschi, D., eds.. 2008. *Mobility, data mining and privacy - geographic knowledge discovery*. New York:Springer.
Gudmundsson, J. and van Kreveld, M.J., 2006. Computing longest duration flocks in trajectory data. *In*: *GIS*. New York:ACM Press, 35–42.
Guo, L., Huang, G., and Ding, Z., 2014. Efficient detection of emergency event from moving object data streams. *In*: *DASFAA*. New York:Springer, 422–437.
Helmi, S. and Banaei-Kashani, F., 2020. Multiscale frequent co-movement pattern mining. *In*: *ICDE*. Washington, DC: IEEE Computer Society, 829–840.
Jeung, H., *et al*., 2008b. Discovery of convoys in trajectory databases. *PVLDB*, 1 (1), 1068–1080.

Jeung, H., Shen, H.T., and Zhou, X., 2008a. Convoy queries in spatio-temporal databases. *In*: *ICDE*. Washington, DC:IEEE Computer Society, 1457–1459.

Kontopoulos, I., *et al*., 2020. Real-time maritime anomaly detection: detecting intentional ais switch-off. *International Journal of Big Data Intelligence*, 7 (2), 2. doi:10.1504/IJBDI.2020.107375

Lan, R., *et al*., 2017. Discovering evolving moving object groups from massive-scale trajectory streams. *In*: *MDM*. Washington, DC:IEEE Computer Society, 256–265.

Li, X., *et al*. 2013. Effective online group discovery in trajectory databases. *IEEE Transactions on Knowledge and Data Engineering*, 25 (12), 2752–2766. doi:10.1109/TKDE.2012.193

Li, Z., *et al*., 2010. Swarm: mining relaxed temporal moving object clusters. *PVLDB*, 3 (1), 723–734.

Matsubara, Y., *et al*., 2014. Fast and exact monitoring of co-evolving data streams. *In*: *ICDM*. Washington, DC: IEEE Computer Society, 390–399.

Orakzai, F., Calders, T., and Pedersen, T.B., 2019. k/2-hop: fast mining of convoy patterns with effective pruning. *PVLDB*, 12 (9), 948–960.

Patroumpas, K., Pelekis, N., and Theodoridis, Y., 2018. On-the-fly mobility event detection over aircraft trajectories. *In*: *SIGSPATIAL/GIS*. New York:ACM Press, 259–268.

Pelekis, N. and Theodoridis, Y., 2014. *Mobility data management and exploration*. New York:Springer.

Ray, C., *et al*., 2019. Heterogeneous integrated dataset for maritime intelligence, surveillance, and reconnaissance. *Data in Brief*, 25, 104141. doi:10.1016/j.dib.2019.104141

Sawas, A., *et al*., 2019. A versatile computational framework for group pattern mining of pedestrian trajectories. *GeoInformatica*, 23 (4), 501–531.

Shein, T.T., Puntheeranurak, S., and Imamura, M., 2020. Discovery of loose group companion from trajectory data streams. *IEEE Access*, 8, 85856–85868. doi:10.1109/ACCESS.2020.2992596

Spiliopoulou, M., *et al*., 2006. MONIC: modeling and monitoring cluster transitions. *In*: *KDD*. New York:ACM Press, 706–711.

Tang, L.A., *et al*., 2012. On discovery of traveling companions from streaming trajectories. *In*: *ICDE*. Washington, DC:IEEE Computer Society, 186–197.

Vieira, M.R., Bakalov, P., and Tsotras, V.J., 2009. On-line discovery of flock patterns in spatio-temporal data. *In*: *GIS*. New York:ACM Press, 286–295.

Weisstein, E.W., n.d.. Jung's theorem. from mathworld – A wolfram web resource. Accessed 13 February 2020, Available from: http://mathworld.wolfram.com/JungsTheorem.html.

Wiratma, L., *et al*., 2019. An experimental evaluation of grouping definitions for moving entities. *In*: *SIGSPATIAL*. New York:ACM Press, 89–98.

Xian, Y., *et al*., 2019. Parallel discovery of trajectory companions from heterogeneous streaming data. *In*: *COMPSAC (1)*. Washington, DC:IEEE Computer Society, 453–462.

Yadamjav, M., *et al*., 2019. Querying continuous recurrent convoys of interest. *In*: *SIGSPATIAL*. New York:ACM Press, 436–439.

Zhang, D., Lee, K., and Lee, I., 2019. Semantic periodic pattern mining from spatio-temporal trajectories. *Information Sciences*, 502, 164–189. doi:10.1016/j.ins.2019.06.035

Zhao, B., *et al*. 2020. A framework for group converging pattern mining using spatiotemporal trajectories. *GeoInformatica*, 24 (4), 745–776. doi:10.1007/s10707-020-00404-z

Zheng, K., *et al*. 2014. Online discovery of gathering patterns over trajectories. *IEEE Transactions on Knowledge and Data Engineering*, 26 (8), 1974–1988. doi:10.1109/TKDE.2013.160

Zheng, Y., *et al*., 2008. Understanding mobility based on GPS data. *In*: *UbiComp*. New York:ACM Press, Vol. 344, 312–321.

Zheng, Y., *et al*., 2009. Mining interesting locations and travel sequences from GPS trajectories. *In*: *WWW*. New York:ACM Press, 791–800.

Zheng, Y., Xie, X., and Ma, W., 2010. Geolife: A collaborative social networking service among user, location and trajectory. *IEEE Data Engineering Bulletin*, 33 (2), 32–39.